SENG 300 Server and Network

The goal of this document is to provide all the documentation potentially needed to use and work with a database server. This first section explains the general workings of the server, network, and database systems. The second section explains how to set up the server, and the third section explains how to run it. A basic understanding of computer networking principles and cryptology is assumed, but I have tried to provide a simplified explanation of some sections for those without. Generally speaking, understanding the TLDR sections is likely enough for most contributors; additional explanations are only provided for curiosity and possible edge cases, so feel free to skip directly to the TLDR sections. Throughout this doc, the users' computers may be referred to as "clients", while the actual database is referred to as the "server".

Dependencies:

The server computer must have SQLite and a JDBC driver installed. Minor changes to default Windows settings are also required. The client computers require nothing extra.

Overview:

On the server, a single port is opened, and the computer constantly listens for TCP/IP connections from clients. Currently on the test server, port 14001 is being used. This can be changed if necessary.

For clients, the server computer is located and connected to using the server's internal IP address and the port number (in this case, port 14001). Connections are always initiated by clients.

Once a connection has been made, data is transferred between the two in the form of a stream of bits, which are interpreted by the computers as a byte array. These transmissions are unencrypted and visible to other devices on the network, and so for security reasons a basic encryption system is also implemented.

Connections are NOT held indefinitely; Right now, the server is only able to handle one client at a time, so connections are only held as long as required for the data transfer, before being broken to allow other clients to connect. In the case that multiple clients attempt to connect to the server at the same time, a queue is formed, and clients are handled one at a time in the order that they arrived. The current queue capacity is set to 100, and can be arbitrarily increased if needed. Any attempts to connect when the queue is full are refused.

As a side note, it seems to be possible to handle multiple connections at once by enabling multithreading on the server side. With multithreading, it might be possible to maintain a single unbroken connection throughout a client's entire session, instead of forming and breaking connections multiple times. This has not been tested, and places higher demands on the server computer. This been avoided for now because of uncertainty on the specs of the computers being used for server hosting. If it becomes necessary, this can be revisited in the future.

TLDR:

Currently, there are two versions of the program: One for the clients (i.e. users) and one for a dedicated server. All of the information for the platform (usernames, passwords, game info, etc…) are held on the server. When the client needs to request/update information, a connection is made to the server over the network, and the necessary data is transferred.

Encryption:

The first time a client computer connects to the server, a new client ID is assigned to the client. Client ID's are tied to the client computers, not the account(s) using that computer. Client ID's are stored in persistent memory (possibly a .txt file of some sort? Hasn't been decided yet), and sent at the start of all future transmissions with the server.

After assigning a new client ID, an encryption key is generated for use between the server and the specific client. First, a shared secret known only to the client and server is created using the Diffie-Hellman Protocol. This shared secret is then used to generate an encryption key for the AES algorithm on GCM mode. On the server side, these keys are stored alongside the associated client IDs in Java's Keystore object, inside of "AESkeys.jks" file. For all future communications with the same client, the keys are reused. Likewise, the client also saves their encryption key for later reuse with the server on a "AESkeys.jks" file. Keystores are encrypted storage, and it takes linear time to retrieve the keys, with the time scaling based on the size of the keystore. So on server and client start up, all premade keys are loaded from the keystores into a HashMap and a temporary variable, respectively, to reduce connection delays.

The most notable feature of AES-GCM encryption is the inclusion of nonces. Normally, when two of the plaintexts are encrypted, the same ciphertext is produced. This is generally considered undesirable, as it allows attackers to pattern match and recognise when the same bits/transmissions are being sent. Nonces (sometimes also referred to as IV, which used to perform similar roles in now outdated AES modes) are randomly generated bits added to every message before AES encryption is performed, so as to ensure that two identical plaintexts will not produce the same ciphertexts. For decryption purposes, the nonce has to be passed along and declared to the other party, and so the first 12 bytes of every encryption transmission includes a copy of the nonce.

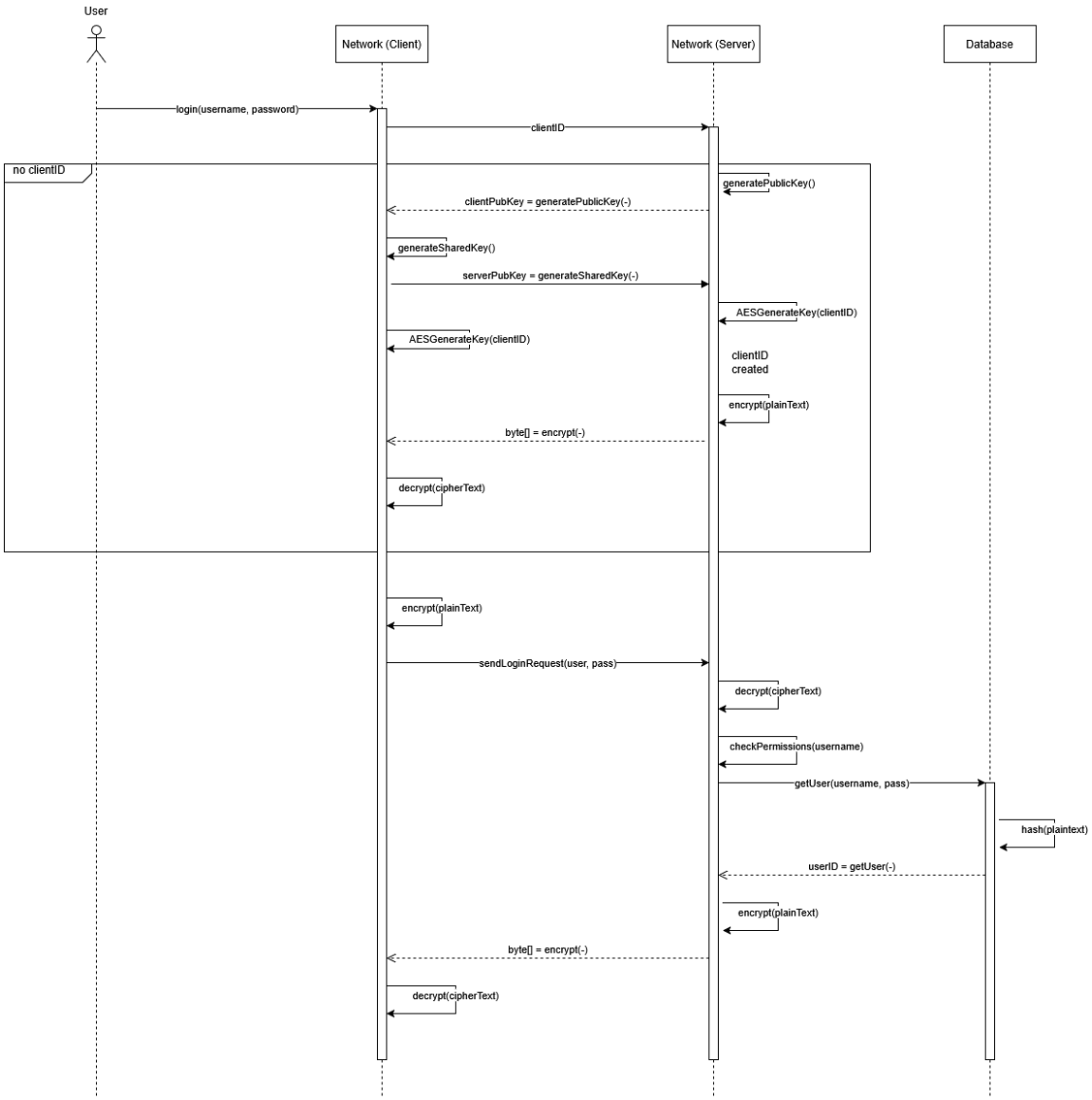
Beyond that, when transmitting raw bytes over the network, the other party has to know where each message starts and ends, so they know when to stop/start listening. In the current setup, each message is preceded by 4 bytes, which indicate the length of the actual message. Combined with AES encryption, the average conversation looks something like this:

First transmission (client sends clientID):

Rest of transmissions:

Where the green bytes represent the transmission length (minus the 4 green bytes), blue represents the nonce, and yellow is the actual contents of the message (in ciphertext).

Here is a sequence diagram illustrating the login process when a new client attempts to connect (credit to Sameet for providing the diagram):



Note: The initial clientID sent by the client is empty, and thus the server creates and assigns the client a new one

TLDR:

Honestly, there's no need to understand how encryption works. All you need to know from this -if anything all- is that messages are encrypted, and client IDs are tied to the computers, NOT user accounts.

(Non)issues:

There are 2 main complications with this setup:

1) Before connecting to the server, clients must manually adjust their code to match the server's IP address

IP addresses are normally assigned by the router; Without knowing what IP is going to be assigned to your server, the clients cannot be configured to connect automatically. One partial workaround for this is using static IP address. IP addresses are normally randomly assigned by the router using the DHCP protocol. If you use a static IP for your server instead (either by changing your server's network settings or adjusting your router settings), you can give your server a fixed IP address that doesn't change. Now you will only have to adjust the client's code the very first time the server goes online, and never again (so long as you stay on the same network).

2) Both the server and client(s) must be on the same network

When a device sends out a packet into the internet, the router will replace the device's IP with a public IP (oftentimes the router's IP) using NAT. To everyone else on a different network, all they can see is the router and not any of the devices actually on it. When you try to directly connect to the server from a different network, the connection will fail because the actual server is "invisible". The IP address assigned to the server is an internal IP address, and only exists within that subnet. Outside of it, the IP is considered unroutable, and all routers will automatically drop packets meant for private IP addresses. Even if it were somehow possible for the packets to reach the server's router without getting dropped, routers still automatically refuse connections initiated from outside because of a lack of NAT mapping.

Both of these problems are only problems for the current school setup, and wouldn't actually be an issue for our fictional scenario. As a real company, with control over our company's internet subscriptions and router, both these problems could easily be solved. With a business-class internet subscription, it would be possible for our router to get a static IP, which would never change (normally, for a non-commercial internet subscription, routers are given random IP addresses by the ISP). With that, client programs could be hard-coded to connect to our router's specific IP address and would never have to manually edit the client-side code. At the same time, with access to our router, we could set up port forwarding to allow connections from other networks. As mentioned earlier, direct connections to devices from different networks are not possible. But it is possible to directly connect to the server's router, which *does* have a

public IP address. Then, with port forwarding enabled on the router, traffic to the router can be redirected to the server, allowing connections from other networks.

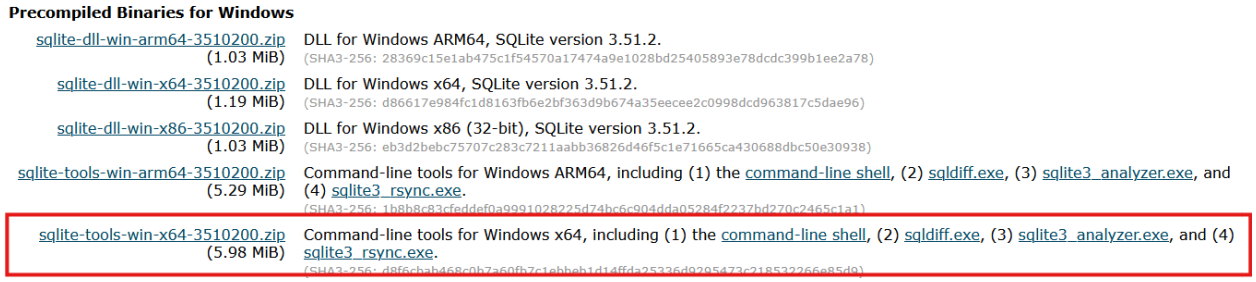
Server Set-up

This guide explains how to set up a Windows computer as a server. This should also work on Linux, but hasn't been tested on it.

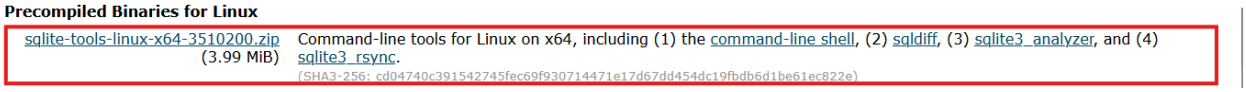
Downloading SQLite:

Download the sqlite-tools for the appropriate OS from https://www.sqlite.org/download.html.

For Windows:



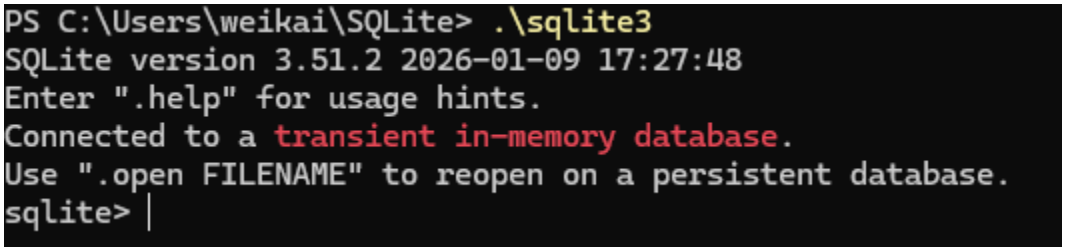
For Linux:



You should have downloaded a .zip file. Go to your desired directory, and create a new folder called "SQLite" (I put mine inside my "c:\users\yourname" folder). Right-click the downloaded .zip and extract it into this folder. Check to make sure there are at least 3 files, "sqldiff.exe", "sqlite3.exe", and "sqlite3_analyzer.exe".

Checking the installation:

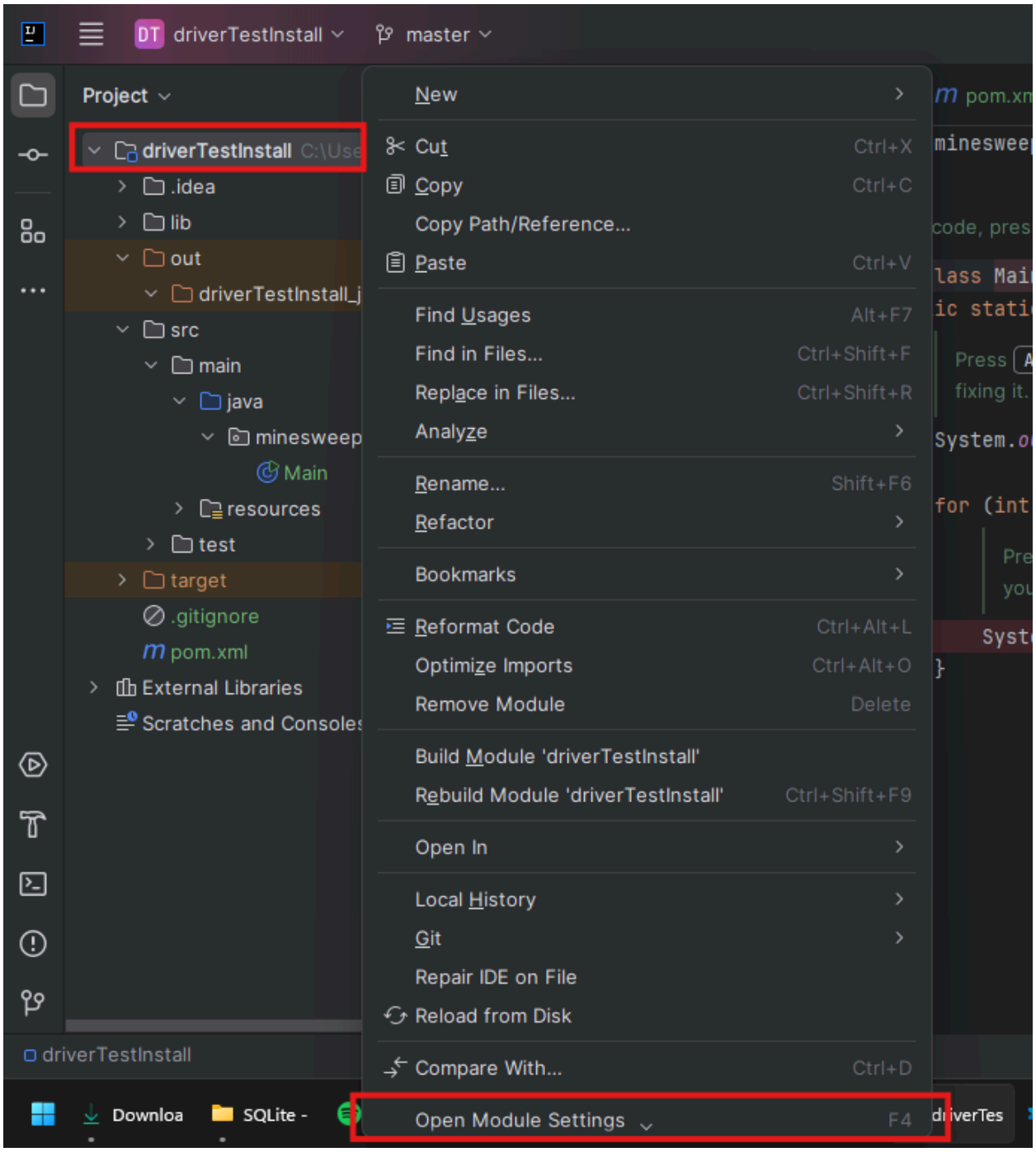
Open up the Windows Command Prompt terminal (or the Linux equivalent), and navigate inside the "SQLite" folder you just created. If you are using Windows, type ".\sqlite3" into the terminal and press enter. Otherwise, if you are using Linux, type "sqlite3". You should see the following:



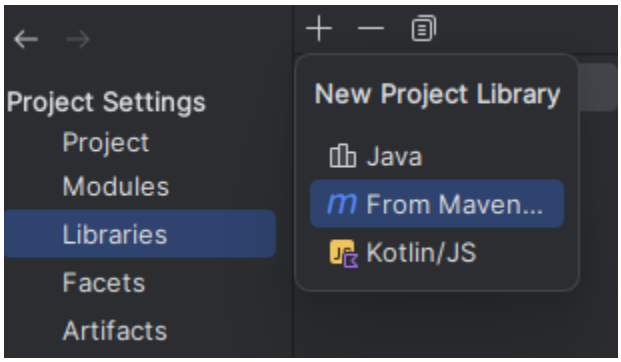
At this point, SQLite has been installed successfully, and you can type ".quit" into the terminal to exit.

Setting up IntelliJ:

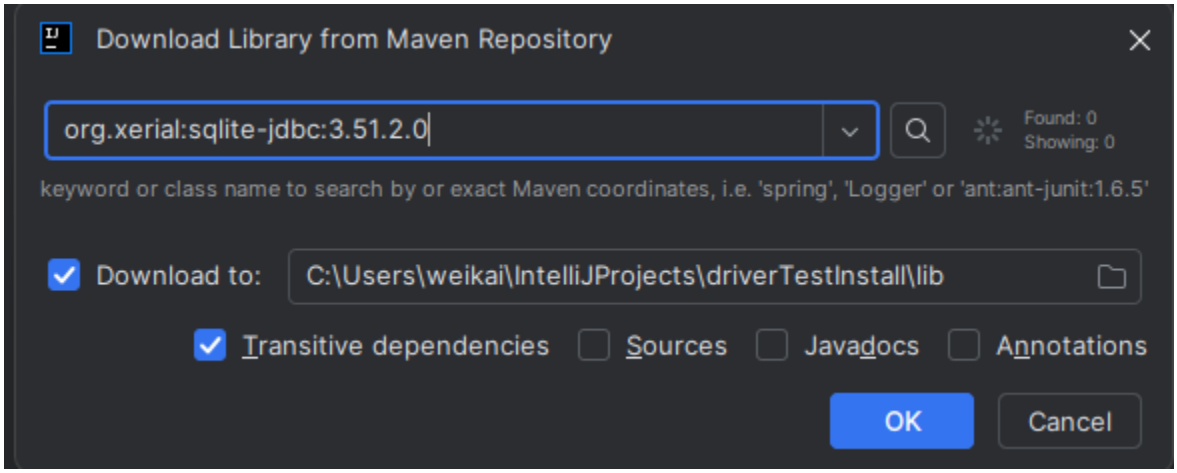
To allow SQL to work with Java, a JDBC driver is required. Open your IntelliJ project, and right-click on the main project folder. Select “Open Module Settings”. Note: I am using the old (and no longer supported) community edition of IntelliJ, so my screenshots might look a little different from yours.



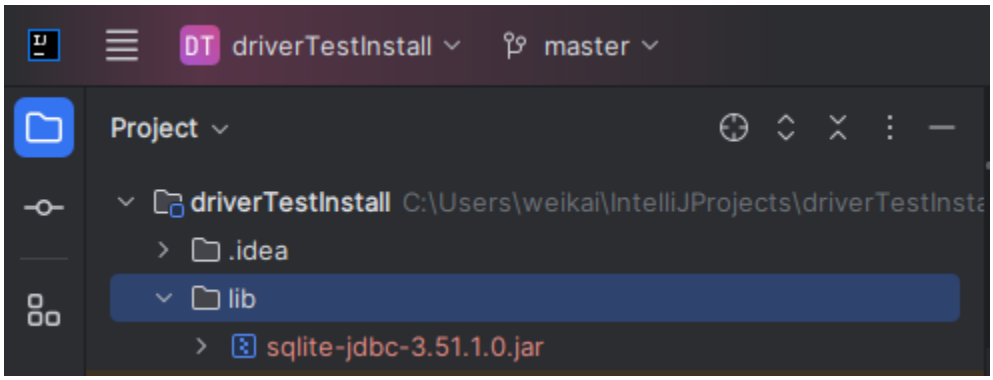
Select the “Libraries” tab on the left, and click the “+” symbol and then “From Maven”. A pop-up will appear.



In the popup, enter “org.xerial:sqlite-jdbc:X.XX.X.X” (subsitute with digits of newest version, which can be found here: https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc), select “transitive dependencies” and “download to”, then click ok. In this example, the newest version is 3.51.2.0.



Some confirmation popups will appear, select ok to confirm.
Inside your project folders, there will now be a “lib” folder (if there wasn’t already), and inside of it you should see the SQLite JDBC driver.

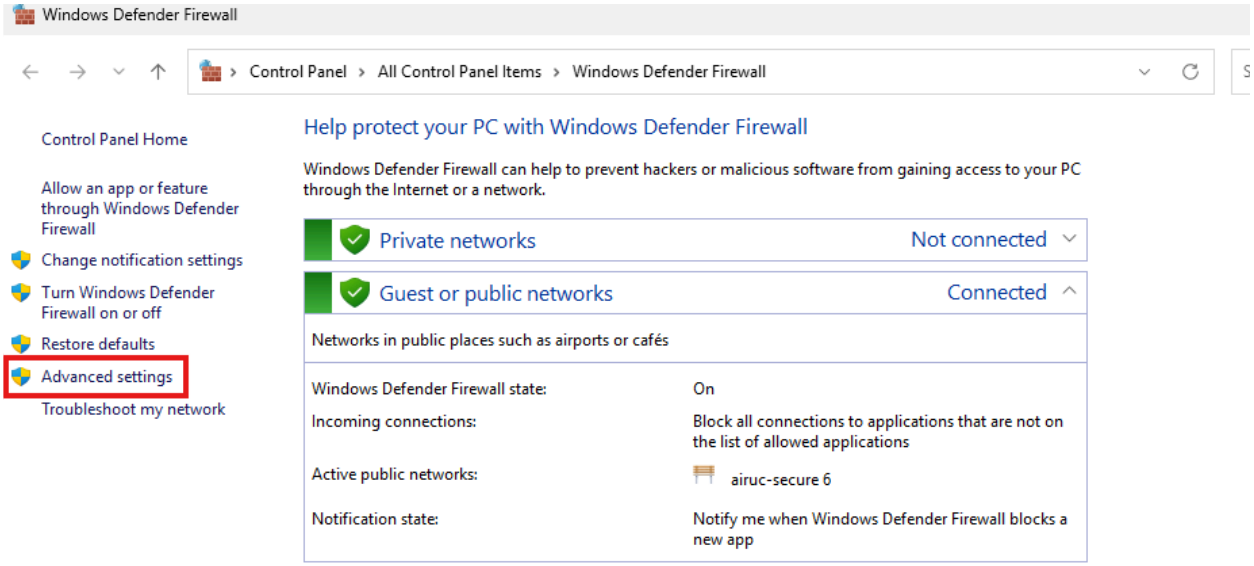


Running the server:

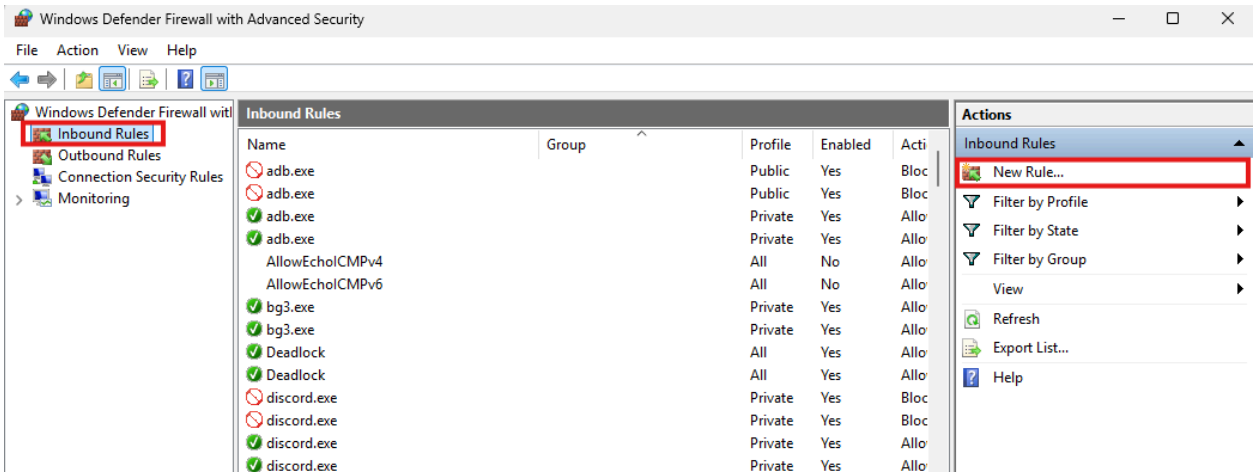
Unfortunately, the server cannot be run from the IntelliJ IDE like a normal program. Before the server can be run, the computer's firewall permissions have to be adjusted, and an artefact created.

Firewall Permissions:

The server listens on a specific port for connections. But for any connections to be made, a hole has to first be opened in the firewall to allow communication. To do this, **admin privileges are required.** Press the Windows key, and search for the Windows Defender Firewall app. Open it, and select the "Advanced Settings" tab.



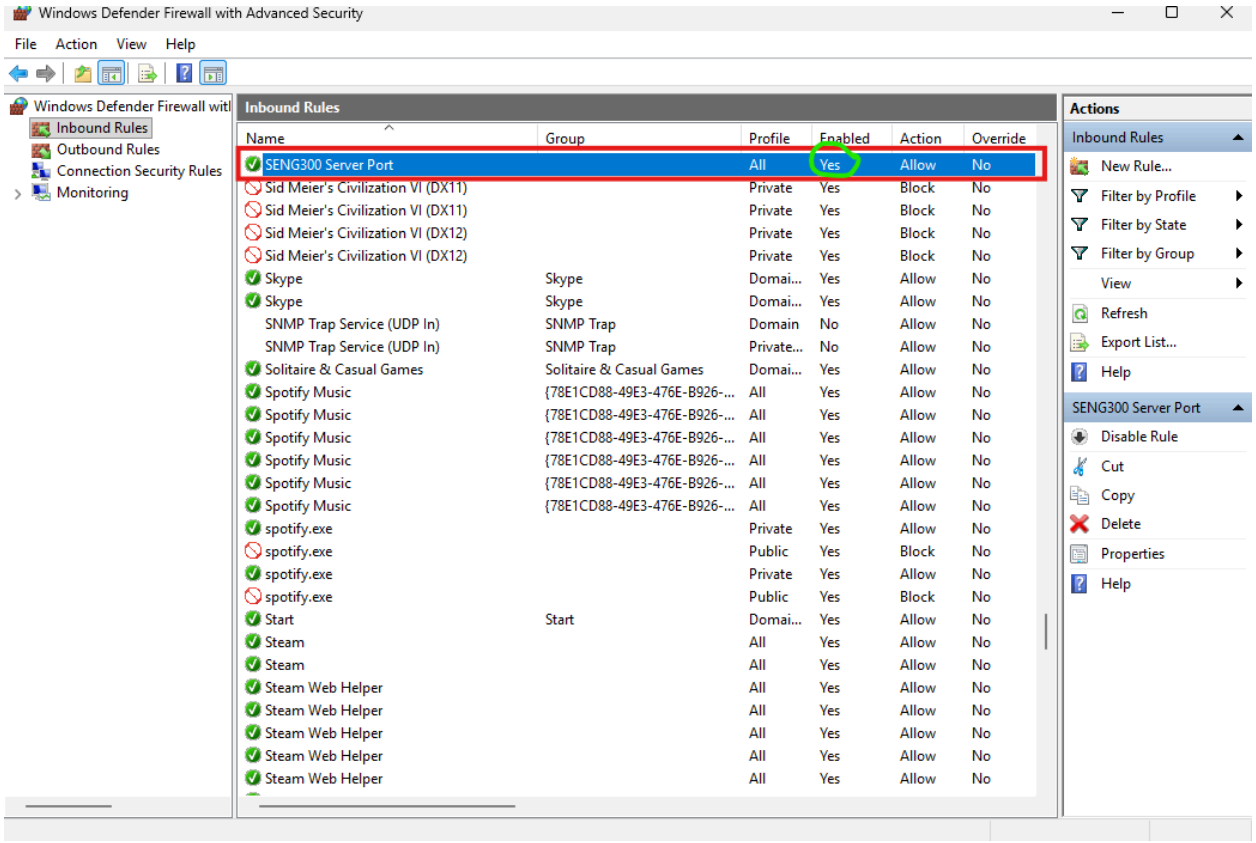
A new pop-up will appear, select the "Inbound Rules" tab and click "New Rule…".



Select the following options:
Rule type: Port

Protocol and Ports: TCP and specific local port. Put in the port number the server is being hosted on. You can check the port number being used inside the Database.java file.
Action: Allow the connection
Profile: Everything
Name: Up to you, choose a name and description that will allow you to find this rule later.

Once that is done, select finish. You should now see your rule in the list. Click on it, and make sure it is enabled.



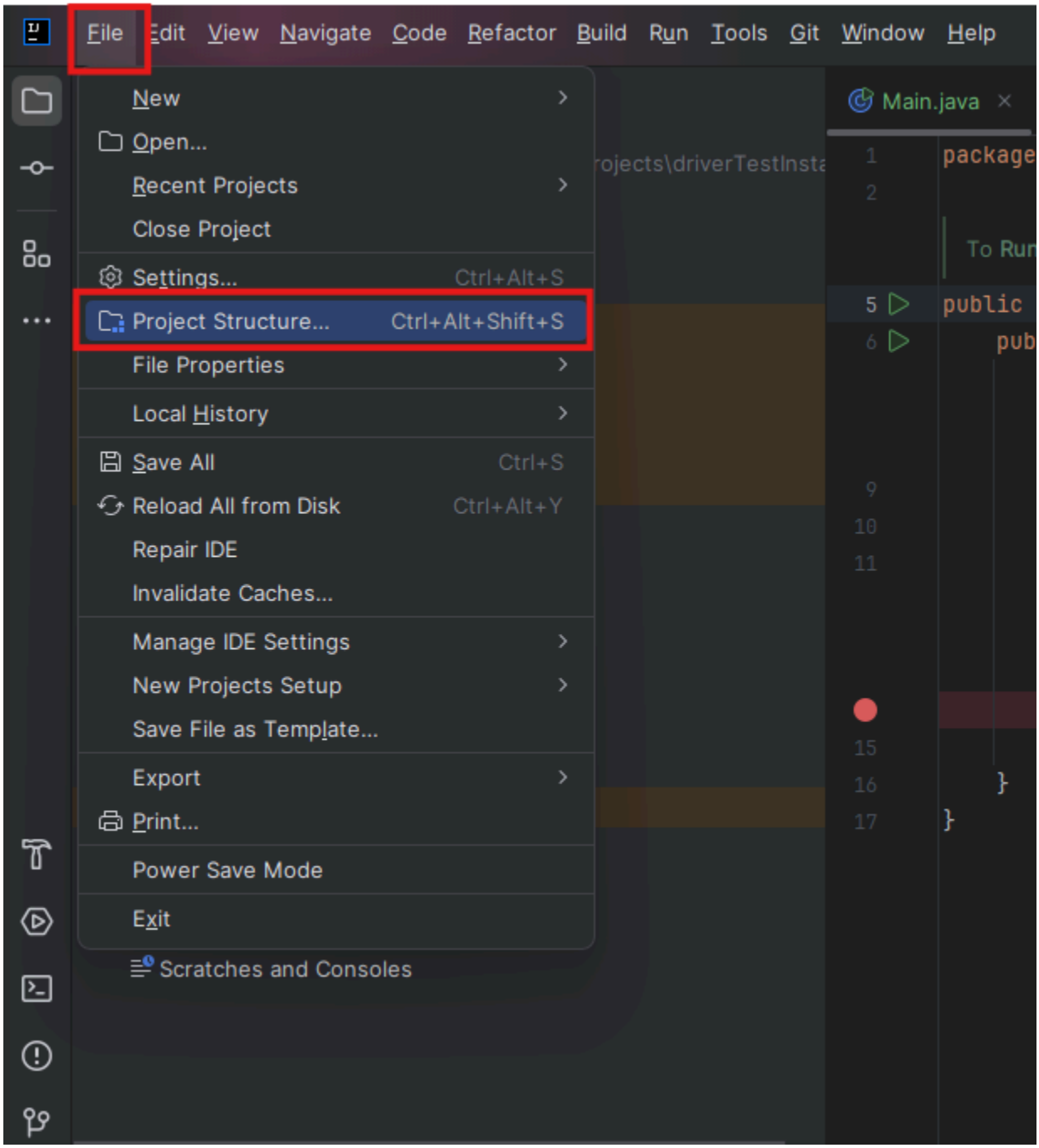
The newly added rule should be at the top of the list, but if you cannot find it you can sort the list by name. It should be enabled by default; if it is not, you can select the rule and enable it from the right sidebar.

Then, select “Outbound Rules” from the left sidebar, and repeat the process. Once that is done, you should have 2 new firewall rules allowing access for the server’s port number. One for inbound, and one for outbound. For security reasons, I recommend disabling these rules when the server is not running.

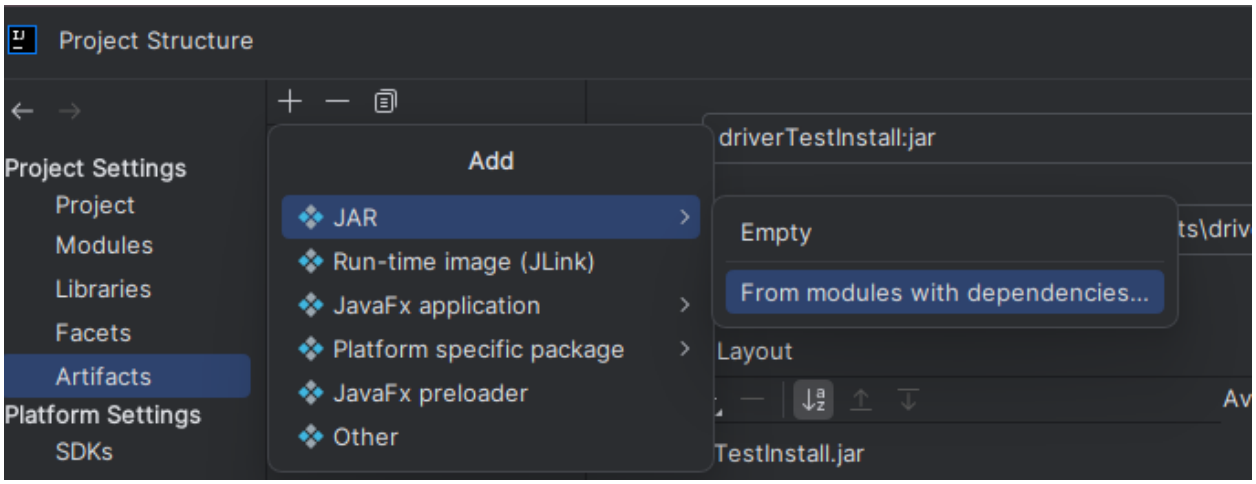
Creating an Artefact:

This program cannot run from inside the IDE. Even with firewall permissions for the port, the IntelliJ IDE doesn’t have access. A workaround for this is to run the project as a .jar from the terminal instead. This next section will cover how to package the project as a .jar (i.e. artifact) and run it.

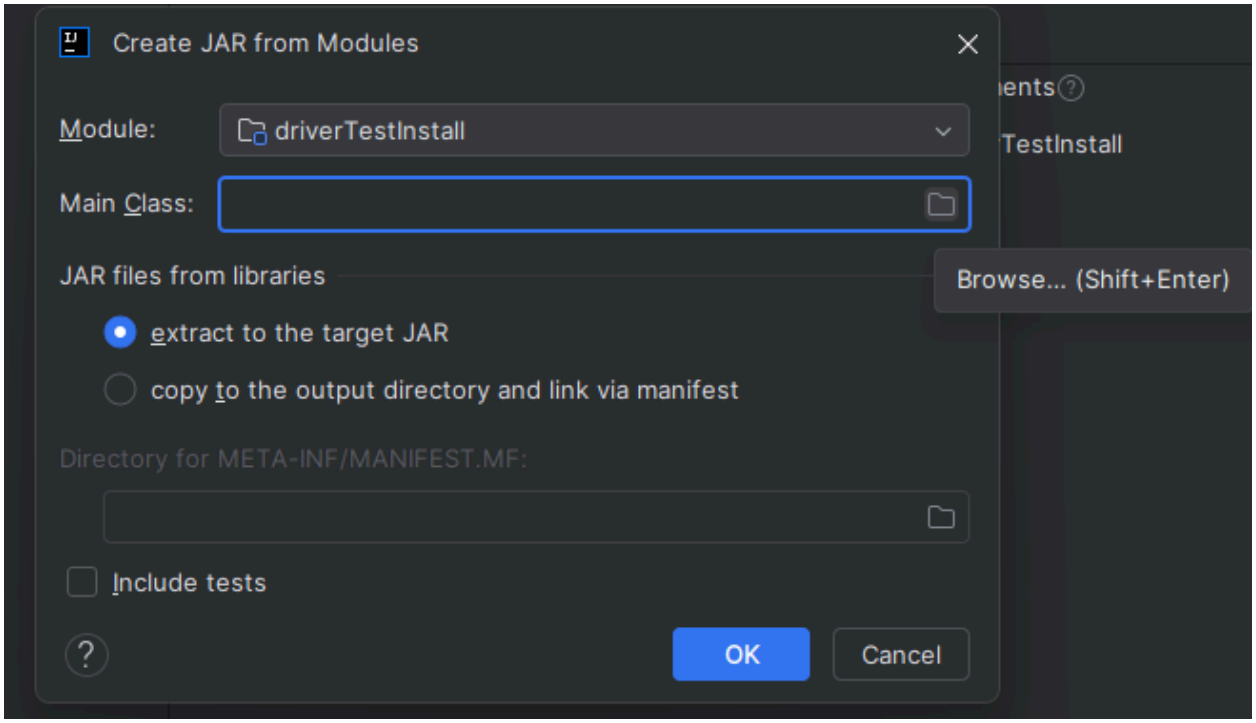
Open the server project files in IntelliJ. Select the “Project Structure” option from the top toolbar.



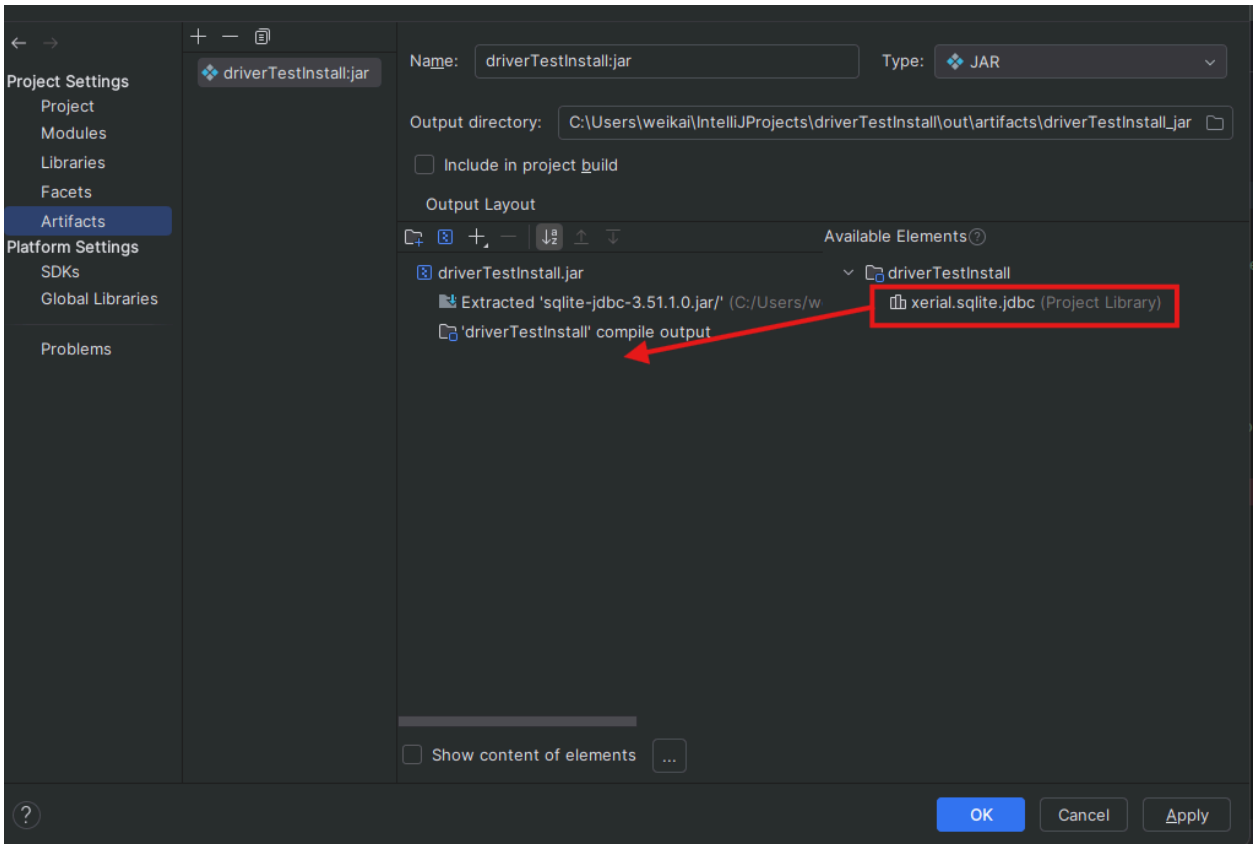
A new pop-up will appear. Select the “Artifacts” tab from the left, and then press the “+” sign. Choose the following options.



A new pop-up will appear. Select the browse option for the Main Class, and choose the method the server launches from from the list that appears.

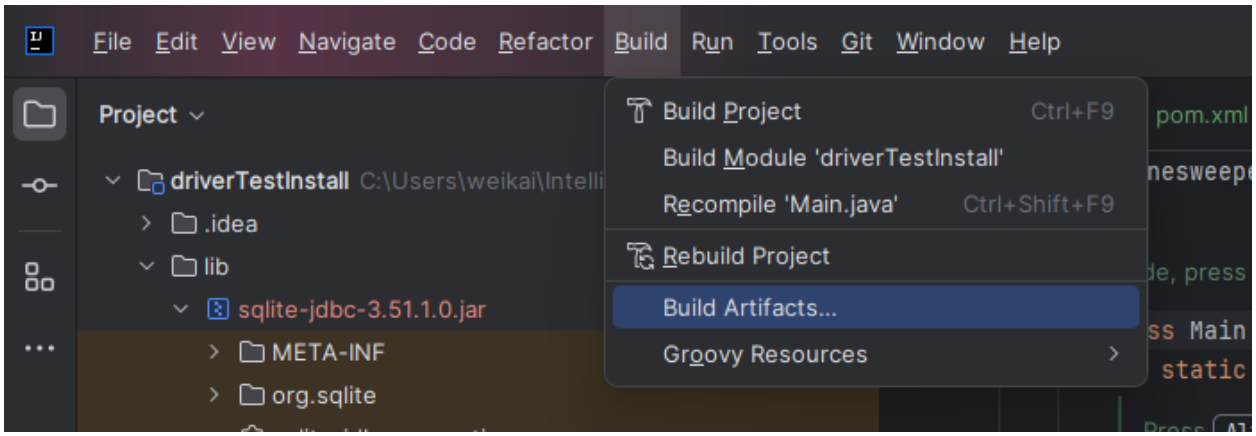


Select Ok. Check the newly created artifact; the JDBC driver might not be included by default. If it’s not, double-click it to add. (It should move from the “Available Elements” section over to the left so it’s under the .jar)

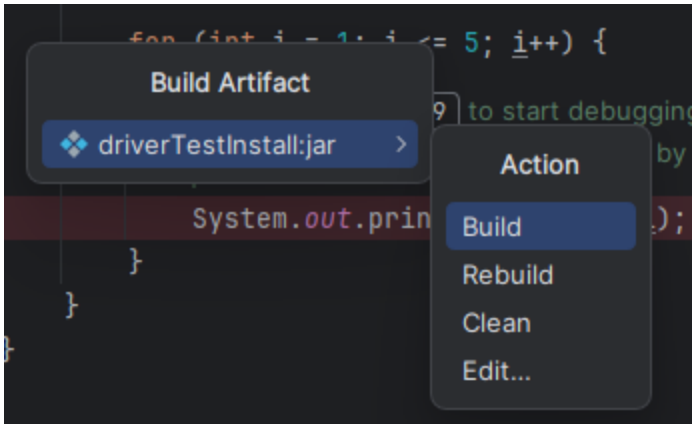


The next step is optional, but I highly recommend changing the file path of the .jar file. In the “Output Directory” field at the top, delete everything after the word “out”. This just removes some unnecessary subfolders for ease of access. Then click “Apply” and “Ok”.

From the top tab, select “Build” and “Build Artifacts”.



A new menu will appear. Select “Build”.



A new “out” subfolder will be created in the project files, with the .jar file inside.

To run the program, open the command terminal and navigate to the directory containing the .jar file. You can also use IntelliJ’s built-in terminal for this. Type “java -jar artifactName.jar” into the terminal to run the program.

The .jar file does NOT automatically update to reflect the changes made in the code. To update the .jar file, select Build > Build Artifacts > Rebuild. If at any point the .jar file is lost/deleted, a new one can be created anytime by choosing the “Build” option instead of “Rebuild”.